ULTRACORE RFT

LABORATORY

Reality Fractal Theory · Stable Invariant Rift Model

RESEARCH DOSSIER

Deterministic Execution Architecture | Runtime Security | Invariant-Preserving Computation

5	4.29B+	0	4
Research Programs	Fuzz Executions	Invariant Violations	Bugs Fixed Upstream

Organization	UltraCore RFT Laboratory | github.com/RFT-SIRM
Focus	Blockchain runtime architecture · deterministic execution · invariant-preserving computation
Engineering	Rust (stable) · libFuzzer · GitHub Actions · Apache 2.0
Validation	Continuous fuzzing 5h 55m daily across all active programs
RFC submitted	anza-xyz/svm — official Agave / Solana repository
Stage	Research laboratory · active prototypes · pre-revenue
Date	July 2026

1. EXECUTIVE SUMMARY

UltraCore RFT Laboratory is an independent research laboratory focused on next-generation deterministic distributed systems. The laboratory investigates blockchain runtime architecture, deterministic execution environments, invariant-preserving computation, transaction scheduling, memory safety, and formal runtime validation.

The scientific foundation of the laboratory consists of two interconnected frameworks: Reality Fractal Theory (RFT), which provides the conceptual and mathematical basis, and the Stable Invariant Rift Model (SIRM), which defines the execution model. All engineering work within the laboratory is an implementation or validation of these principles.

The laboratory currently operates five research programs spanning protocol security, runtime scheduling, Layer-1 implementation, and Solana SVM integration. All programs are backed by continuous automated fuzzing and regression testing. In the course of this work, the laboratory has identified and fixed critical bugs in the Agave SVM execution layer, with formal findings submitted to the official anza-xyz/svm repository.

What the laboratory is not

UltraCore RFT Laboratory is not a consumer product company, a DeFi protocol, or a token launch vehicle. It is an engineering-first research organization operating at the infrastructure layer. The work it produces — verified runtime components, scheduler implementations, formal invariant specifications — is the type of foundational work that blockchain ecosystems depend on but rarely produce in public with full verification evidence.

Audience

- Blockchain infrastructure engineers evaluating runtime contributions
- Protocol architects reviewing Solana SVM and Layer-1 designs
- Venture funds considering early-stage infrastructure research
- Strategic partners in the Solana and broader blockchain ecosystem

2.1 Reality Fractal Theory (RFT)

Reality Fractal Theory is the conceptual framework that unifies mathematics, deterministic computation, distributed execution, economics, and protocol architecture under a common set of principles. The central observation of RFT is that consistent systems — whether physical, mathematical, or computational — exhibit invariant structure across scale and concurrency.

Applied to distributed systems, RFT asserts that correctness is not a property to be verified after execution but a structural constraint that must hold at every step. This shifts the design paradigm from probabilistic consistency guarantees (as seen in most existing blockchains) toward deterministic, mathematically enforced invariants.

2.2 Stable Invariant Rift Model (SIRM)

SIRM is the execution model derived from RFT principles. It specifies a runtime in which mathematical invariants are not external checks but intrinsic properties of every state transition. Under SIRM, an operation either preserves all defined invariants or is rejected atomically — there is no intermediate state.

The four core SIRM invariants, as implemented in the Rift-L1-Blockchain program:

```
I1 (Economic balance): total_supply = total_base_sum + (global_field × participants)
I2 (Mint/burn ledger): total_minted >= total_burned
supply = minted - burned
I3 (Dust control): dust_accumulator < participants
I4 (Debt ceiling): effective_balance >= -(supply / (10 × participants))
```

Each invariant has a precise mathematical statement, a reason for existence, a defined failure mode, a corresponding fuzz assertion, and a regression test. This is the standard the laboratory applies to all research programs.

2.3 UltraCore Rift — Engineering Implementation

UltraCore Rift is the engineering architecture that implements RFT and SIRM principles in production-quality Rust code. It defines how deterministic state machines, invariant-preserving operations, and conflict-aware scheduling work together to form a coherent execution environment. The UltraCore-RFT repository serves as the architectural reference and documentation hub for the entire laboratory.

Determinism	Identical inputs always produce identical outputs — no hidden state, no non-deterministic paths
Atomicity	Every operation is all-or-nothing. Partial state transitions do not exist
Invariance	Mathematical invariants are checked after every single operation, not periodically
Verifiability	All behavior is reproducible from deterministic seeds; state can be replayed

The following five programs are not independent projects. They are interconnected research and engineering efforts, each addressing a different layer of the UltraCore RFT architecture. Together they form a complete stack from protocol theory down to SVM execution internals.

Program	Layer	Status	Validation
UltraCore-RFT	Architecture / Theory	Active	Documentation
Rift-Network	Solana Protocol	Audited	14 findings addressed
Rift-L1-Blockchain	L1 Runtime	Active	256M+ ops · 0 violations
agave-abiv2-memory-contexts	SVM Security (Agave)	Active	4.29B+ exec · 0 violations
agave-rift-scheduler	SVM Scheduling (Agave)	Active	91M exec/run · 0 violations

3.1 UltraCore-RFT — Architectural Reference

Research goal	Establish the theoretical foundation and architectural specification for deterministic distributed systems based on RFT and SIRM
Engineering obj.	Produce living documentation: ARCHITECT.md, RFT_MATHEMATICAL_FOUNDATIONS.md, RFT_DEVELOPMENT_STRATEGY.md
Status	Active — continuously updated as other programs produce findings
Why it matters	Provides the shared conceptual language and design constraints used by all other programs. Without this, the research programs would be isolated engineering efforts rather than a coherent laboratory

3.2 Rift-Network — Solana Protocol Implementation

Research goal	Implement RFT economic model and SIRM invariants on the Solana/Anchor stack. Validate protocol correctness through formal security audit
Engineering obj.	Production-quality Anchor program with on-chain SIRM invariant enforcement and complete security audit coverage
Status	Audited. 14 findings identified across severity levels, all addressed. Licensed Apache 2.0
Invariant	total_supply = total_base_sum + (global_field × p) — enforced on-chain after every instruction
Why it matters	Demonstrates that SIRM invariants can be enforced inside an existing production blockchain (Solana). Acts as the bridge between the laboratory's theoretical work and the Solana ecosystem

3.3 Rift-L1-Blockchain — Standalone Runtime

Research goal	Build a standalone Layer-1 blockchain runtime where all four SIRM invariants are enforced at the execution core level, verified through large-scale fuzz testing
Engineering obj.	CoreState implementation with checked arithmetic, atomic invariant enforcement, and a fuzz harness capable of running billions of randomized operations
Status	Active. 256M+ operations verified, 0 invariant violations, 0 crashes. Fuzzing runs 5h 55m daily
Why it matters	Provides the purest implementation of SIRM — no external protocol constraints, no smart contract layer. This is the mathematical core of the UltraCore architecture

Platform	Verified ops/sec	Per 5h 55m run
GitHub CI (ubuntu x86, 2 vCPU)	~2,000,000	~42 billion
Apple M1 (arm64, verified)	8,530,712	~181 billion
32-core server (projected)	50–60M	~1.0–1.3 trillion

3.4 agave-abiv2-memory-contexts — SVM Security Research

Research goal	Investigate the correctness of per-CPI-frame writable permission isolation in Agave SVM ABIv2 memory context layer
Engineering obj.	Identify, reproduce, fix, and regression-test permission leakage bugs in the Agave execution environment
Status	Active. Critical bug found and fixed. 4.29B+ fuzz executions, 0 violations. RFC open in anza-xyz/svm
Why it matters	CPI permission leakage is a structural vulnerability affecting any Solana protocol that uses nested Cross-Program Invocations. This covers virtually all DeFi protocols on Solana. The TVL exposure is measured in billions of dollars

Bug found: snapshot.entries.clear() — permission leakage across CPI frames

The original Agave implementation called snapshot.entries.clear() before re-snapshotting on every call to update_account_permissions() within the same CPI frame. If a single frame issued more than one permission update, the rollback entry for the first account was destroyed. On frame pop(), only the last-touched account was restored — all earlier accounts retained their modified writable permission across CPI frame boundaries.

Fix: HashSet-based first-occurrence snapshot

A HashSet tracks which region_index values have already been snapshotted in the current frame. Only the first occurrence of each account is recorded. Subsequent permission updates do not overwrite the original snapshot. On pop(), the true original value is always restored.

Metric	Value
Total fuzz executions	4,294,967,296+
Execution speed (CI)	~421,000 exec/s
Invariant violations	0
Coverage stabilised at	cov: 53 · ft: 166

3.5 agave-rift-scheduler — SVM Scheduling Research

Research goal	Research conflict-aware transaction scheduling for Agave SVM with formal invariant guarantees and bounded retry semantics
Engineering obj.	Hybrid scheduler implementation with heat-based hotspot tracking, generation-based backoff, max_retry_count cap, and four scheduling invariants
Status	Active. 3 bugs found and fixed. 91M exec/run, 0 violations, 15 unit tests passing. RFC open in anza-xyz/svm
Why it matters	Transaction scheduling is a direct throughput constraint for every Solana validator. Correctness bugs in the scheduler cause silent transaction loss — a category of failure that is difficult to detect without formal verification

Bug	Description	Impact	Status
Dead deferred queue	Deferred txs pushed but never retried	Silent transaction loss	Fixed
Zero-cost bypass	cost=0 txs bypassed conflict detection	Invariant violation	Fixed
Fuzzer invariant gap	Backoff chains exceeded drain limit	Stuck transactions	Fixed

Four scheduling invariants, verified through continuous fuzzing:

```
I1 (Accounting): scheduled + deferred + dropped <= scanned (per pass)
I2 (Generation): summary.generation > 0 (after every pass)
I3 (Monotonicity): scheduler_passes increments on every schedule() call
I4 (Drain bound): deferred queue reaches zero within 8192 drain passes
```

All UltraCore RFT Laboratory research programs are validated through a unified multi-layer methodology. The methodology is designed to provide strong experimental evidence of correctness without overstating formal guarantees.

4.1 Validation Layers

Layer	Tooling	What it verifies	Coverage
Unit tests	cargo test	Regression, edge cases, bug fixes	15+ tests · all passing
Property tests	Manual seeds	Random inputs, 100+ seeds per property	All properties hold
Differential tests	Buggy vs fixed	Bug reproducibility and fix correctness	All diffs confirmed
Fuzz testing	libFuzzer	All invariants under arbitrary inputs	4B+ exec · 0 violations
Long-run fuzz	GitHub CI/CD	Stability over time, rare edge cases	5h 55m daily, all green
Format + lint	rustfmt + clippy	Code quality, -D warnings on every push	All green

4.2 Continuous Fuzzing Infrastructure

Each active research program maintains a dedicated libFuzzer harness. The harness generates randomized inputs, constructs valid operation sequences, and asserts all defined invariants after every step. Coverage typically stabilises within the first 100,000 executions, indicating the harness has explored the reachable state graph for the given input size constraints.

Corpus management	Persistent corpus per program. REDUCE phase minimises corpus entries after coverage stabilisation
Mutation strategy	libFuzzer default: bit flips, byte substitutions, splicing, insertion, deletion — guided by coverage
State generation	Arbitrary-derived structs (via the arbitrary crate) generate full scheduler configs and operation sequences
Termination	max_total_time = 21300s (5h 55m) on schedule; 60s smoke run on every push
Drain validation	8192 additional passes after input stops — guarantees I4 (deferred queue convergence)

4.3 Why Deterministic Validation Matters for Layer-1 Systems

Probabilistic correctness guarantees — the standard in most existing blockchains — are insufficient for execution layer components. A scheduler that loses transactions 0.01% of the time may pass all probabilistic tests while causing systematic value loss at scale. A memory permission system that

leaks across CPI frames only under specific call sequences may appear correct under normal workloads while enabling targeted exploits.

Deterministic validation — invariant enforcement after every operation, backed by billions of randomized executions — does not prove correctness in the formal sense, but it provides a qualitatively different level of assurance than unit tests or manual code review alone. The absence of a single invariant violation across 4.29 billion executions is a meaningful engineering claim.

5.1 Implemented and Validated

Contribution	Program	Evidence
CPI permission rollback (HashSet snapshot)	memory-contexts	4.29B+ exec · 0 violations · RFC submitted
Dead deferred queue fix	scheduler	91M exec/run · regression test passing
Zero-cost conflict bypass fix	scheduler	15 unit tests · fuzz verified
Fuzzer drain invariant (8192 passes)	scheduler	crash-4c00ef fixed · verified locally and CI
SIRM CoreState (4 invariants)	Rift-L1	256M+ ops · 0 violations · 5h55m daily
Security audit — 14 findings	Rift-Network	All findings addressed · Apache 2.0
Scheduler invariants I1–I4	scheduler	Fuzz-encoded · passing daily
Heat-based hotspot tracking	scheduler	Unit tests · integrated in fuzz harness
Generation-based backoff (cap 2^8)	scheduler	Mathematically bounded · verified

5.2 Planned (not yet implemented)

- Integration of memory-contexts fix into anza-xyz/agave (requires upstream review response)
- Criterion benchmark suite: scheduler throughput before/after, hotspot-heavy workloads
- Formal RFC / Draft PR against anza-xyz/agave with corpus and invariant documentation
- Mainnet deployment of Rift-Network
- Validator integration testing for Rift-L1-Blockchain
- Formal verification (model checking / theorem proving) — outside current scope

5.3 RFC Status

A formal issue has been opened in the official anza-xyz/svm repository describing the per-CPI-frame writable permission leakage, the root cause, the fix, and the fuzz evidence. This is the standard channel for contributing to the Agave execution layer. A response from the Anza engineering team is pending.

6. CURRENT DEVELOPMENT STAGE

UltraCore RFT Laboratory is currently at the research prototype stage. All five programs have working implementations. Three programs run continuous automated validation on GitHub Actions. One program has completed a formal security audit. One program has an open RFC in an official upstream repository.

Maturity level	Research prototype — working code, continuous validation, no production deployment
Revenue	Pre-revenue. The laboratory does not currently generate revenue
Team	Solo founder / engineering lead. All research and implementation by one person
Infrastructure	GitHub Actions CI/CD, libFuzzer, Rust stable, public repositories
Upstream presence	Open RFC in anza-xyz/svm · Apache 2.0 license on all programs
Documentation	README, ARCHITECTURE.md, invariant documentation, engineering review docs per program

What "engineering-first laboratory" means in practice

- Every claim is backed by reproducible evidence — fuzz logs, test output, or documented findings
- No features are described as implemented unless working code exists and passes CI
- All bugs found are documented with reproduction cases and regression tests
- The laboratory does not publish timelines it cannot commit to engineering resources for
- Code is Apache 2.0 — auditable, forkable, and verifiable by any third party

UltraCore RFT Laboratory is exploring its first external funding round. The purpose of this funding is to accelerate the development of the laboratory's research programs and transition from a solo-founder research operation to a small, focused engineering team.

This is not a token sale, a software license, or a consumer product offering. Investment in the laboratory is investment in the continued development of deterministic execution research and its practical application to Solana SVM and next-generation Layer-1 architecture.

7.1 Intended Use of Funds

Engineering team	Hire 2–3 senior Rust / systems engineers to parallelize research programs
Protocol research	Formal specification of SIRM, independent mathematical review, research publication
Runtime implementation	Production-quality implementation of UltraCore Rift as a standalone execution environment
Validator testing	Deploy Rift-L1 on testnet infrastructure with real validator load
Formal verification	Engage formal verification specialists for select invariants (model checking / Lean)
Benchmarking	Criterion suite comparing scheduler and memory contexts performance against production Agave
Security review	Third-party audit of all active research programs
Layer-1 development	Mainnet preparation for Rift-L1-Blockchain and Rift-Network

7.2 Round Structure

Round type	Seed / Strategic
Target raise	$3,000,000
Instrument	SAFE with Token Warrant (open to discussion)
Use of funds	As described in 7.1 — engineering, research, validation, team
Current investors	None. This is the first external round
TGE	No date set. Technical validation and partnership establishment first

7.3 What investors can evaluate today

- Five public Apache 2.0 repositories with full commit history
- Continuous fuzzing evidence: 4.29B+ executions, 0 invariant violations
- Documented bug findings with reproduction cases and fixes
- Open RFC in anza-xyz/svm — visible engagement with the Agave ecosystem
- Security audit report with 14 documented findings on Rift-Network

- Architecture documentation and mathematical foundations in public repositories

The long-term goal of UltraCore RFT Laboratory is to build a deterministic blockchain architecture grounded in RFT and SIRM — an execution environment where mathematical invariants are structural properties of the runtime, not post-hoc checks. This is a multi-year engineering objective, not a near-term product launch.

Phase 1	**Foundation**	Standalone verified components across all five programs. All core invariants implemented and fuzz-validated. Security audit completed on Rift-Network. RFC submitted to anza-xyz/svm.	**Complete**
Phase 2	**Agave Integration**	Integrate memory-contexts and scheduler fixes into anza-xyz/agave. Replace research mocks with real Agave types. Produce Criterion benchmarks before/after. Publish formal Draft PR with full evidence package.	**In progress**
Phase 3	**Testnet & Benchmarks**	Deploy Rift-L1-Blockchain on testnet with real validator infrastructure. Quantify throughput improvement from scheduler fixes at validator scale. Engage formal verification for select SIRM invariants.	**Planned**
Phase 4	**Production & Ecosystem**	Rift-Network mainnet deployment. Rift-L1-Blockchain production launch. DeFi ecosystem on SIRM-verified runtime. Bridge infrastructure with Solana and Ethereum.	**Planned**

What success looks like

Success for UltraCore RFT Laboratory is not defined by market capitalization or token price. It is defined by the quality and impact of engineering contributions: fixes accepted into production Agave, SIRM invariants independently verified, Rift-L1 running on real validator infrastructure, and a team capable of continuing this research at scale. The financial upside for investors follows from these engineering milestones, not the reverse.

Repository hub	https://github.com/RFT-SIRM
Memory contexts	https://github.com/RFT-SIRM/agave-abiv2-memory-contexts
Scheduler	https://github.com/RFT-SIRM/agave-rift-scheduler
L1 Blockchain	https://github.com/RFT-SIRM/Rift-L1-Blockchain
Rift Network	https://github.com/RFT-SIRM/Rift-Network
Research Hub	https://github.com/RFT-SIRM/UltraCore-RFT
Contact	@Mercurius_Maximus (Telegram)

RFC	https://github.com/anza-xyz/svm/issues (open)